

LAPORAN PRAKTIKUM

Struktur Data

“Algoritma Sorting”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan yang berjudul "Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)" dengan baik.

Laporan ini disusun sebagai bagian dari tugas praktikum Struktur Data yang bertujuan untuk memahami konsep dasar algoritma pengurutan (sorting) serta penerapannya dalam pemrograman. Algoritma sorting merupakan salah satu materi penting dalam ilmu komputer karena digunakan untuk mengatur data sehingga lebih mudah dikelola, dicari, dan dianalisis. Dalam laporan ini dibahas tiga algoritma pengurutan yang umum digunakan, yaitu **Bubble Sort**, **Selection Sort**, dan **Insertion Sort**, beserta implementasinya dalam program.

Melalui pembuatan program dan penyusunan laporan ini, penulis memperoleh pemahaman yang lebih mendalam mengenai cara kerja masing-masing algoritma, kelebihan dan kekurangannya, serta perbandingan efisiensi dalam proses pengurutan data. Diharapkan laporan ini dapat menjadi sarana pembelajaran yang bermanfaat bagi penulis maupun pembaca yang ingin mempelajari algoritma sorting.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan, baik dari segi isi maupun penyajiannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi para pembaca.

Padang, Agustus 2026

Abdul Jabbar

DAFTAR ISI

BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	2
2.1 Pengertian	2
2.2 jenis jenis algoritma sort dan cara kerjanya	2
2.3 Praktikum Pekan 8	4
2.3.1 Buble Sort	4
2.3.2 MergeSort	8
2.3.3 QuicSort	9
2.3.4 ShellSort	11
2.3.5 Laporan tugas_2511533021	13
BAB III PENUTUP	15
3.1 Kesimpulan	15
3.2 Saran	15
<u>DAFTAR PUSTAKA</u>	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menyebabkan jumlah data yang diolah oleh komputer terus meningkat. Data tersebut dapat berupa angka, teks, nama, nilai, maupun informasi lainnya yang perlu dikelola secara efektif dan efisien. Dalam pengolahan data, salah satu proses yang sangat penting adalah pengurutan data (*sorting*). Pengurutan data bertujuan untuk menyusun data berdasarkan urutan tertentu, seperti dari nilai terkecil ke terbesar, terbesar ke terkecil, atau berdasarkan urutan alfabet. Dengan data yang terurut, proses pencarian, analisis, dan pengelolaan informasi dapat dilakukan dengan lebih cepat dan mudah.

1.2 Tujuan

1. Memahami konsep dasar algoritma sorting sebagai metode untuk mengurutkan data secara sistematis.
2. Mengetahui cara kerja berbagai algoritma sorting, seperti *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*.
3. Mempelajari proses pengurutan data berdasarkan urutan tertentu, baik secara ascending maupun descending.
4. Mengembangkan kemampuan berpikir logis dan analitis dalam menyelesaikan permasalahan pengolahan data.
5. Mengimplementasikan algoritma sorting dalam program komputer untuk meningkatkan efisiensi pengelolaan data.

1.3 Manfaat

1. Mempermudah proses pencarian dan pengolahan data karena data telah tersusun secara teratur.
2. Meningkatkan efisiensi kerja program dalam mengelola data dalam jumlah kecil maupun besar.
3. Membantu pengguna memahami prinsip dasar algoritma dan struktur data dalam ilmu komputer.
4. Melatih kemampuan pemrograman serta pemecahan masalah secara sistematis dan terstruktur.
5. Menjadi dasar untuk mempelajari algoritma yang lebih kompleks, seperti *Merge Sort*, *Quick Sort*, dan algoritma pencarian (*searching*).
6. Memudahkan penerapan pengurutan data pada berbagai aplikasi, seperti sistem akademik, basis data, sistem informasi, dan aplikasi bisnis.

BAB II PEMBAHASAN

2.1 Pengertian

Algoritma sort adalah metode atau langkah-langkah logis untuk mengurutkan sejumlah data. Data yang diurutkan dapat berupa angka, huruf, nama siswa, nilai barang, dan lain-lain.

2.2 jenis jenis algoritma sort dan cara kerjanya

1. Buble Sort adalah algoritma pengurutan sederhana yang bekerja dengan cara membandingkan elemen secara berpasangan, lalu menukarnya jika urutannya salah.
 - Bandingkan dua elemen yang berdekatan.
 - Jika elemen kiri lebih besar, lakukan pertukaran.
 - Ulangi hingga tidak ada pertukaran.
 - Lambat untuk data besar ($O(n^2)$).
 - Mudah dipahami dan diimplementasikan.

2. Selection sort mencari elemen terkecil dalam array, kemudian menempatkannya di posisi awal.
 - Cari elemen terkecil dari seluruh list.
 - Tukar dengan elemen indeks pertama.
 - Ulangi untuk indeks berikutnya.
 - Lebih sedikit pertukaran data.
3. Insertion Sort bekerja dengan cara menyisipkan elemen ke posisi yang benar pada bagian list yang sudah terurut.
 - Mulai dari elemen kedua.
 - Pindahkan elemen ke posisi yang tepat di subarray terurut.
 - Ulangi hingga akhir array.
4. Merge Sort menggunakan teknik *divide and conquer* dengan membagi list menjadi dua bagian, mengurutkan masing-masing, lalu menggabungkannya.
 - Bagi array menjadi dua bagian.
 - Urutkan kedua bagian secara rekursif.
 - Gabungkan dengan proses merge.
5. Quick Sort memilih satu elemen sebagai pivot, kemudian mempartisi array menjadi dua bagian berdasarkan pivot tersebut.
 - Pilih pivot.
 - Bagi data menjadi lebih kecil dan lebih besar dari pivot.
 - Rekursif ke dua bagian tersebut.
6. Heap Sort menggunakan struktur data *heap* untuk mengurutkan data.
 - Bangun struktur heap dari array.
 - Ambil elemen terbesar/terkecil.
 - Rekonstruksi heap hingga semua elemen terurut.

2.3 Praktikum Pekan 8

Pada praktikum Pekan 8, mahasiswa Informatika Universitas Andalas mempelajari selection sort dengan membuat dan mengimplementasikan kode program menggunakan aplikasi Eclipse IDE.

2.3.1 Buble Sort

```
30 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.border.EmptyBorder;
20
21 public class BubleSortGUI_2511533021 extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private int[] array_3021;
25     private JLabel[] labelArray_3021;
26     private JButton stepButton_3021, resetButton_3021, setButton_3021;
27     private JTextField inputField_3021;
28     private JPanel panelArray_3021;
29     private JTextArea stepArea_3021;
30     private JPanel contentPane_3021;
31
32
33     private int i_3021 = 1, j_3021;
34     private boolean sorting_3021 = false;
35     private int stepCount_3021 = 1;
36
37
38
39
40     public static void main(String[] args) {
41         EventQueue.invokeLater(new Runnable() {
42             public void run() {
43                 try {
44                     BubleSortGUI_2511533021 frame_3021 = new BubleSortGUI_2511533021();
45                     frame_3021.setVisible(true);
46                 } catch (Exception e_3021) {
47                     e_3021.printStackTrace();
48                 }
49             }
50         });
51     }
52
53     /**
54      * Create the frame.
55      */
56     public BubleSortGUI_2511533021() {
57         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
58         setBounds(100, 100, 450, 300);
59         contentPane_3021 = new JPanel();
60         contentPane_3021.setBorder(new EmptyBorder(5, 5, 5, 5));
61         setContentPane(contentPane_3021);
62         setTitle("Insertion Sort langkah per langkah");
63         setSize(750, 400);
64         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
65         setLocationRelativeTo(null);
66         setLayout(new BorderLayout());
67
68         // panel input
69         JPanel inputPanel_3021 = new JPanel(new FlowLayout());
70         inputField_3021 = new JTextField(30);
71         setButton_3021 = new JButton("Set Array");
72         inputPanel_3021.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
```

```

73     inputPanel_3021.add(inputField_3021);
74     inputPanel_3021.add(setButton_3021);
75
76     // panel array visual
77     panelArray_3021 = new JPanel();
78     panelArray_3021.setLayout(new FlowLayout());
79
80     // panel kontrol
81     JPanel controlPanel_3021 = new JPanel();
82     stepButton_3021 = new JButton("Langkah Selanjutnya");
83     resetButton_3021 = new JButton("Reset");
84     resetButton_3021.setEnabled(false);
85     controlPanel_3021.add(stepButton_3021);
86     controlPanel_3021.add(resetButton_3021);
87
88     // Area teks untuk log langkah langkah
89     stepArea_3021 = new JTextArea(8, 60);
90     stepArea_3021.setEditable(false);
91     stepArea_3021.setFont(new Font("Monospaced", Font.PLAIN, 14));
92     JScrollPane scrollPane_3021 = new JScrollPane(stepArea_3021);
93
94     // tambahkan panel ke frame
95     add(inputPanel_3021, BorderLayout.NORTH);
96     add(panelArray_3021, BorderLayout.CENTER);
97     add(controlPanel_3021, BorderLayout.SOUTH);
98     add(scrollPane_3021, BorderLayout.EAST);
99
100    // Event set array
101    setButton_3021.addActionListener(e_3021 -> setArrayFromInput_3021());
102
103    // Event langkah selanjutnya
104    stepButton_3021.addActionListener(e_3021 -> performStep_3021());
105
106    // Event Reset

```

```

106    // Event Reset
107    resetButton_3021.addActionListener(e_3021 -> reset_3021());
108
109    }
110
111    private void setArrayFromInput_3021() {
112        String text_3021 = inputField_3021.getText().trim();
113        if (text_3021.isEmpty())
114            return;
115        String[] parts_3021 = text_3021.split(",");
116        array_3021 = new int[parts_3021.length];
117        try {
118            for (int k_3021 = 0; k_3021 < parts_3021.length; k_3021++) {
119                array_3021[k_3021] =
120                    Integer.parseInt(parts_3021[k_3021].trim());
121            }
122        } catch (NumberFormatException e_3021) {
123            JOptionPane.showMessageDialog(
124                this,
125                "Masukkan hanya angka yang dipisahkan koma!",
126                "Error",
127                JOptionPane.ERROR_MESSAGE);
128        }
129        return;
130    }
131
132    i_3021 = 0;
133    j_3021 = 0;
134
135    stepCount_3021 = 1;
136    sorting_3021 = true;
137    stepButton_3021.setEnabled(true);
138    stepArea_3021.setText("");
139    panelArray_3021.removeAll();

```

```

140    labelArray_3021 = new JLabel[array_3021.length];
141    for (int k_3021 = 0; k_3021 < array_3021.length; k_3021++) {
142        labelArray_3021[k_3021] =
143            new JLabel(String.valueOf(array_3021[k_3021]));
144        labelArray_3021[k_3021].setFont(
145            new Font("Arial", Font.BOLD, 24));
146        labelArray_3021[k_3021].setOpaque(true);
147        labelArray_3021[k_3021].setBackground(Color.WHITE);
148        labelArray_3021[k_3021].setBorder(
149            BorderFactory.createLineBorder(Color.BLACK));
150        labelArray_3021[k_3021].setPreferredSize(
151            new Dimension(50, 50));
152        labelArray_3021[k_3021].setHorizontalAlignment(
153            SwingConstants.CENTER);
154        panelArray_3021.add(labelArray_3021[k_3021]);
155    }
156
157    panelArray_3021.revalidate();
158    panelArray_3021.repaint();
159
160    private void performStep_3021() {
161
162        if (!sorting_3021 || j_3021 >= array_3021.length - 1) {
163            sorting_3021 = false;
164            stepButton_3021.setEnabled(false);
165            JOptionPane.showMessageDialog(
166                this,
167                "Sorting selesai!");
168            return;
169        }
170
171        resetHighlights_3021();
172
173

```



```

174     StringBuilder stepLog_3021 =
175         new StringBuilder();
176
177     labelArray_3021[j_3021]
178         .setBackground(Color.CYAN);
179
180     labelArray_3021[j_3021 + 1]
181         .setBackground(Color.CYAN);
182
183     if (array_3021[j_3021]
184         > array_3021[j_3021 + 1]) {
185
186         // Swap
187
188         int temp_3021 =
189             array_3021[j_3021];
190
191         array_3021[j_3021] =
192             array_3021[j_3021 + 1];
193
194         array_3021[j_3021 + 1] =
195             temp_3021;
196
197         labelArray_3021[j_3021]
198             .setBackground(Color.RED);
199
200         labelArray_3021[j_3021 + 1]
201             .setBackground(Color.RED);
202
203         stepLog_3021.append("Langkah ")
204             .append(stepCount_3021)
205             .append(": Menukar elemen ke-")
206             .append(j_3021)

```

```

206             .append(j_3021)
207             .append(" ")
208             .append(array_3021[j_3021 + 1])
209             .append(" dengan ke-")
210             .append(j_3021 + 1)
211             .append(" ")
212             .append(array_3021[j_3021])
213             .append(")\n");
214
215     } else {
216
217         stepLog_3021.append("Langkah ")
218             .append(stepCount_3021)
219             .append(": Tidak ada pertukaran antara ke-")
220             .append(j_3021)
221             .append(" dan ke-")
222             .append(j_3021 + 1)
223             .append(")\n");
224
225     }
226
227     stepLog_3021.append("Hasil: ")
228         .append(arrayToString_3021(array_3021))
229         .append("\n\n");
230
231     stepArea_3021.append(
232         stepLog_3021.toString());
233
234     updateLabels_3021();
235
236     j_3021++;
237
238     if (j_3021 >= array_3021.length - i_3021 - 1) {
239         i_3021 = 0;

```

```

241         i_3021++;
242     }
243
244     stepCount_3021++;
245
246     if (i_3021 >= array_3021.length - 1) {
247         sorting_3021 = false;
248         stepButton_3021.setEnabled(false);
249
250         JOptionPane.showMessageDialog(
251             this,
252             "Sorting selesai!");
253     }
254 }
255
256
257
258
259
260 private void updateLabels_3021() {
261     for (int k_3021 = 0; k_3021 < array_3021.length; k_3021++) {
262         labelArray_3021[k_3021].setText(String.valueOf(array_3021[k_3021]));
263     }
264 }
265
266 private void resetHighlights_3021() {
267     for (JLabel label_3021 : labelArray_3021) {
268         label_3021.setBackground(Color.WHITE);
269     }
270 }
271
272 private void reset_3021() {
273     inputField_3021.setText("");
274
275     private void reset_3021() {
276         inputField_3021.setText("");
277         panelArray_3021.removeAll();
278         panelArray_3021.revalidate();
279         panelArray_3021.repaint();
280         stepArea_3021.setText("");
281         stepButton_3021.setEnabled(false);
282         sorting_3021 = false;
283         i_3021 = 0;
284         j_3021 = 0;
285         stepCount_3021 = 1;
286     }
287
288     private String arrayToString_3021(int[] arr_3021) {
289         StringBuilder sb_3021 = new StringBuilder();
290         for (int k_3021 = 0; k_3021 < arr_3021.length; k_3021++) {
291             sb_3021.append(arr_3021[k_3021]);
292             if (k_3021 < arr_3021.length - 1)
293                 sb_3021.append(", ");
294         }
295         return sb_3021.toString();
296     }
297 }

```

1. main() method utama untuk pintu masuk program
2. BubleSortGUI_2511533021() constructor, method khusus untuk inialisasi GUI.
3. setArrayFromInput_3021() method untuk ambil input dari user, ubah jadi array angka, lalu tampilkan di panel
4. performStep_3021() method untuk jalankan bubble sort langkah demi langkah.
5. updateLabels_3021() method untuk update tampilan label sesuai isi array terbaru.
6. resetHighlights_3021() method untuk mengembalikan warna label ke putih.
7. reset_3021() method untuk reset aplikasi ke kondisi awal.

8. `arrayToString_3021()` method untuk ubah array jadi string rapi (misalnya `[5,3,1]` $\rightarrow$ `"5, 3, 1"`).

2.3.2 MergeSort

```
1 package pekan8_2511533021;
2
3 public class MergeSort_2511533021 {
4
5     void merge_3021(int arr_3021[], int l_3021, int m_3021, int r_3021) {
6
7         int n1_3021 = m_3021 - l_3021 + 1;
8         int n2_3021 = r_3021 - m_3021;
9
10        int L_3021[] = new int[n1_3021];
11        int R_3021[] = new int[n2_3021];
12
13
14        for (int i_3021 = 0; i_3021 < n1_3021; ++i_3021)
15            L_3021[i_3021] = arr_3021[l_3021 + i_3021];
16
17        for (int j_3021 = 0; j_3021 < n2_3021; ++j_3021)
18            R_3021[j_3021] = arr_3021[m_3021 + 1 + j_3021];
19
20        int i_3021 = 0, j_3021 = 0;
21        int k_3021 = l_3021;
22        while (i_3021 < n1_3021 && j_3021 < n2_3021) {
23            if (L_3021[i_3021] <= R_3021[j_3021]) {
24                arr_3021[k_3021] = L_3021[i_3021];
25                i_3021++;
26            } else {
27                arr_3021[k_3021] = R_3021[j_3021];
28                j_3021++;
29            }
30
31            k_3021++;
32        }
33
34        while (i_3021 < n1_3021) {
35            arr_3021[k_3021] = L_3021[i_3021];
36
37            i_3021++;
38            k_3021++;
39        }
40
41        while (j_3021 < n2_3021) {
42            arr_3021[k_3021] = R_3021[j_3021];
43            j_3021++;
44            k_3021++;
45        }
46
47        void sort(int arr_3021[], int l_3021, int r_3021) {
48
49            if (l_3021 < r_3021) {
50
51                // Find the middle point
52                int m_3021 = (l_3021 + r_3021) / 2;
53
54
55                sort(arr_3021, l_3021, m_3021);
56                sort(arr_3021, m_3021 + 1, r_3021);
57
58                merge_3021(arr_3021, l_3021, m_3021, r_3021);
59            }
60        }
61
62
63        static void printArray(int arr_3021[]) {
64            int n_3021 = arr_3021.length;
65            for (int i_3021 = 0; i_3021 < n_3021; ++i_3021)
66                System.out.print(arr_3021[i_3021] + " ");
67            System.out.println();
68        }
69
70    }
71
72    public static void main(String args_3021[]) {
73
74        int arr_3021[] = {12, 11, 13, 5, 6, 7};
75        System.out.println("Sebelum terurut");
76        printArray(arr_3021);
77        MergeSort_2511533021 ob_3021 = new MergeSort_2511533021();
78        ob_3021.sort(arr_3021, 0, arr_3021.length - 1);
79        System.out.println("\nSesudah Terurut menggunakan Merge Sort");
80        printArray(arr_3021);
81    }
```

1. `main()` method utama, pintu masuk program. Menyiapkan array contoh, cetak sebelum sorting, panggil `sort()`, lalu cetak hasil sesudah sorting.
2. `MergeSort_2511533021()` ini bukan constructor karena class tidak punya constructor khusus, jadi otomatis pakai default.
3. `merge_3021()` method untuk menggabungkan dua bagian array (kiri dan kanan) yang sudah terurut menjadi satu array besar terurut.
4. `sort()` method rekursif untuk membagi array jadi dua bagian, urutkan masing-masing, lalu gabungkan dengan `merge_3021`.
5. `printArray()` method untuk menampilkan isi array ke console, supaya bisa lihat hasil sebelum dan sesudah sorting.

2.3.3 QuicSort

```

1 package pekan8_2511533021;
2
3 public class QuicSort_2511533021 {
4
5     // method swap
6     static void swap_3021(int[] arr_3021, int i_3021, int j_3021) {
7
8         int temp_3021 = arr_3021[i_3021];
9         arr_3021[i_3021] = arr_3021[j_3021];
10        arr_3021[j_3021] = temp_3021;
11    }
12
13    // method median of three
14    static void medianOfThree_3021(int[] arr_3021,
15        int low_3021,
16        int high_3021) {
17
18        int mid_3021 =
19            low_3021 + (high_3021 - low_3021) / 2;
20
21        if (arr_3021[low_3021] > arr_3021[mid_3021]) {
22            swap_3021(arr_3021, low_3021, mid_3021);
23        }
24
25        if (arr_3021[low_3021] > arr_3021[high_3021]) {
26            swap_3021(arr_3021, low_3021, high_3021);
27        }
28
29        if (arr_3021[mid_3021] > arr_3021[high_3021]) {
30            swap_3021(arr_3021, mid_3021, high_3021);
31        }
32
33        swap_3021(arr_3021, mid_3021, high_3021);
34    }
35
36    static int partition_3021(int[] arr_3021,
37        int low_3021,
38        int high_3021) {
39
40        medianOfThree_3021(
41            arr_3021,
42            low_3021,
43            high_3021);
44
45        int pivot_3021 = arr_3021[high_3021];
46
47        int i_3021 = (low_3021 - 1);
48
49        for (int j_3021 = low_3021;
50            j_3021 <= high_3021 - 1;
51            j_3021++) {
52
53            if (arr_3021[j_3021] < pivot_3021) {
54
55                i_3021++;
56
57                swap_3021(
58                    arr_3021,
59                    i_3021,
60                    j_3021);
61            }
62        }
63
64        swap_3021(
65            arr_3021,
66            i_3021 + 1,
67            high_3021);
68
69        return (i_3021 + 1);
70    }
71
72 }
73

```

```

76 static void quickSort_3021(int[] arr_3021,
77     int low_3021,
78     int high_3021) {
79
80     if (low_3021 < high_3021) {
81
82         int pi_3021 =
83             partition_3021(
84                 arr_3021,
85                 low_3021,
86                 high_3021);
87
88         quickSort_3021(
89             arr_3021,
90             low_3021,
91             pi_3021 - 1);
92
93         quickSort_3021(
94             arr_3021,
95             pi_3021 + 1,
96             high_3021);
97     }
98 }
99
100 // method print array
101 public static void printArr_3021(int[] arr_3021) {
102
103     for (int i_3021 = 0;
104         i_3021 < arr_3021.length;
105         i_3021++) {
106
107         System.out.print(
108             arr_3021[i_3021] + " ");
109     }
110
111     System.out.println();

```

```

110
111     System.out.println();
112 }
113
114 // main method
115 public static void main(String[] args) {
116
117     int[] arr_3021 =
118         {10, 7, 8, 9, 1, 5};
119
120     int N_3021 = arr_3021.length;
121
122     System.out.print(
123         "Data sebelum diurutkan : ");
124
125     printArr_3021(arr_3021);
126
127     quickSort_3021(
128         arr_3021,
129         0,
130         N_3021 - 1);
131
132     System.out.print(
133         "Data setelah Quick Sort : ");
134
135     printArr_3021(arr_3021);
136 }

```

1. swap_3021() method untuk menukar posisi dua elemen dalam array.
2. medianOfThree_3021() method untuk memilih pivot dengan teknik *median of three* (ambil nilai tengah dari elemen pertama, tengah, dan terakhir, lalu jadikan pivot).
3. partition_3021() method untuk membagi array berdasarkan pivot: elemen lebih kecil dari pivot dipindahkan ke kiri, elemen lebih besar ke kanan. Mengembalikan posisi pivot baru.
4. quickSort_3021() method rekursif untuk menjalankan algoritma Quick Sort:
 1. Tentukan pivot dengan partition_3021().

2. Urutkan bagian kiri.
3. Urutkan bagian kanan.
5. printArr_3021() method untuk menampilkan isi array ke console.
6. main() method utama, pintu masuk program. Membuat array contoh, cetak sebelum sorting, panggil quickSort_3021(), lalu cetak hasil sesudah sorting.

2.3.4 ShellSort

```

1 package pekan8_2511533021;
2
3 public class ShellSort_2511533021 {
4
5     public static void shellSort_3021(int[] A_3021) {
6         int n_3021 = A_3021.length;
7         int gap_3021 = n_3021 / 2;
8
9         while (gap_3021 > 0) {
10
11             for (int i_3021 = gap_3021; i_3021 < n_3021; i_3021++) {
12
13                 int temp_3021 = A_3021[i_3021];
14                 int j_3021 = i_3021;
15
16                 while ((j_3021 >= gap_3021 && A_3021[j_3021 - gap_3021] > temp_3021) {
17                     A_3021[j_3021] = A_3021[j_3021 - gap_3021];
18                     j_3021 = j_3021 - gap_3021;
19                 }
20                 A_3021[j_3021] = temp_3021;
21             }
22             gap_3021 = gap_3021 / 2;
23         }
24     }
25
26     public static void main(String[] args) {
27
28         int[] data_3021 = {5, 10, 4, 6, 8, 9, 7, 2, 1, 5};
29
30         System.out.print("Sebelum: ");
31         printArray_3021(data_3021);
32
33         shellSort_3021(data_3021);
34
35         System.out.print("Setelah (ShellSort): ");
36         printArray_3021(data_3021);
37     }
38
39     public static void printArray_3021(int[] arr_3021) {
40
41         for (int i_3021 : arr_3021)
42             System.out.print(i_3021 + " ");
43         System.out.println();
44     }
45 }

```

shellSort_3021() method utama untuk menjalankan algoritma Shell Sort. Fungsinya:

- Tentukan jarak awal (gap) = panjang array / 2.
- Lakukan perbandingan dan penyisipan elemen dengan jarak tersebut.
- Kurangi gap menjadi setengah, ulangi proses sampai gap = 1.
- Hasil akhirnya array terurut.

`main()` method utama, pintu masuk program. Fungsinya:

- Membuat array contoh {3, 10, 4, 6, 8, 9, 7, 2, 1, 5}.
- Cetak array sebelum sorting.
- Panggil `shellSort_3021()` untuk mengurutkan.
- Cetak array sesudah sorting.

`printArray_3021()` → method untuk menampilkan isi array ke console. Fungsinya:

- Loop semua elemen array.
- Cetak angka dengan spasi.
- Tambahkan baris baru di akhir.

2.3.5 Laporan tugas_2511533021

```
1 package pekan8_2511533021;
2 import java.util.Scanner;
3
4 class Lagu {
5     String judul;
6     String penyanyi;
7     int durasi;
8
9     Lagu(String judul, String penyanyi, int durasi) {
10         this.judul = judul;
11         this.penyanyi = penyanyi;
12         this.durasi = durasi;
13     }
14 }
15
16 public class Sorting_2511533021 {
17
18     static Lagu[] dataLagu_3021 = new Lagu[20];
19     static int jumlahData_3021 = 0;
20
21     static void inputData_3021() {
22         dataLagu_3021[0] = new Lagu("Mio Cristo Piange Diamanti", "Maneskin", 270);
23         dataLagu_3021[1] = new Lagu("La Rumba Del Perdon", "Carlos Vives", 252);
24         dataLagu_3021[2] = new Lagu("La Parla", "Calle 13", 190);
25         dataLagu_3021[3] = new Lagu("Perfect", "Ed Sheeran", 263);
26         dataLagu_3021[4] = new Lagu("Photographie", "Ed Sheeran", 258);
27         dataLagu_3021[5] = new Lagu("Happier", "Olivia Rodrigo", 175);
28         dataLagu_3021[6] = new Lagu("Believer", "Imagine Dragons", 204);
29
30         jumlahData_3021 = 7;
31     }
32
33     static void tampilData_3021() {
34         for (int i = 0; i < jumlahData_3021; i++) {
35             System.out.println((i + 1) + " : "
36
37                 + dataLagu_3021[i].judul + " - "
38                 + dataLagu_3021[i].durasi + " detik");
39         }
40     }
41
42     static void quickSort_3021(int low, int high) {
43         if (low < high) {
44             int pi = partition_3021(low, high);
45             quickSort_3021(low, pi - 1);
46             quickSort_3021(pi + 1, high);
47         }
48     }
49
50     static int partition_3021(int low, int high) {
51         int pivot = dataLagu_3021[high].durasi;
52         int i = low - 1;
53
54         for (int j = low; j < high; j++) {
55             if (dataLagu_3021[j].durasi < pivot) {
56                 i++;
57
58                 Lagu temp = dataLagu_3021[i];
59                 dataLagu_3021[i] = dataLagu_3021[j];
60                 dataLagu_3021[j] = temp;
61             }
62         }
63
64         Lagu temp = dataLagu_3021[i + 1];
65         dataLagu_3021[i + 1] = dataLagu_3021[high];
66         dataLagu_3021[high] = temp;
67
68         return i + 1;
69     }
70
71     public static void main(String[] args) {
72         Scanner input = new Scanner(System.in);
73
74         inputData_3021();
75
76         System.out.println("=== Sorting Playlist NIM: 2511533021 ===");
77         System.out.println("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
78         int pilihan = input.nextInt();
79
80         System.out.println("\nData Sebelum Sorting:");
81         tampilData_3021();
82
83         if (pilihan == 2) {
84             quickSort_3021(0, jumlahData_3021 - 1);
85
86             System.out.println("\nData Setelah Quick Sort (Durasi Asc):");
87             tampilData_3021();
88         } else {
89             System.out.println("\nTugas ini hanya mengimplementasikan Quick Sort.");
90         }
91
92         input.close();
93     }
94 }
```

Quick Sort dipilih karena merupakan salah satu algoritma pengurutan yang efisien dan banyak digunakan dalam pemrograman. Algoritma ini bekerja dengan metode divide and conquer, yaitu membagi data menjadi beberapa bagian berdasarkan pivot kemudian mengurutkannya secara rekursif. Pada rata-rata kasus, Quick Sort memiliki kompleksitas waktu $O(n \log n)$ sehingga lebih cepat dibandingkan beberapa algoritma sorting sederhana

seperti Bubble Sort atau Selection Sort. Selain itu, implementasinya cukup mudah untuk mengurutkan data berdasarkan durasi lagu. Oleh karena itu, Quick Sort cocok digunakan untuk mengurutkan playlist lagu dengan jumlah data yang cukup banyak.

Kompleksitas nya

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n^2)$

Space Complexity : $O(\log n)$ (rekursif)

Data sebelum dan sesudah

```
=== Sorting Playlist NIM: 2511533021 ===  
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):  
2  
  
Data Sebelum Sorting:  
1. Mio Cristo Piange Diamanti - 270 detik  
2. La Rumba Del Perdon - 252 detik  
3. La Perla - 196 detik  
4. Perfect - 263 detik  
5. Photograph - 258 detik  
6. Happier - 175 detik  
7. Believer - 204 detik  
  
Data Setelah Quick Sort (Durasi Asc):  
1. Happier - 175 detik  
2. La Perla - 196 detik  
3. Believer - 204 detik  
4. La Rumba Del Perdon - 252 detik  
5. Photograph - 258 detik  
6. Perfect - 263 detik  
7. Mio Cristo Piange Diamanti - 270 detik
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting merupakan teknik yang digunakan untuk mengurutkan data berdasarkan kriteria tertentu sehingga data menjadi lebih terstruktur dan mudah dikelola. Pada praktikum ini dilakukan implementasi algoritma Quick Sort untuk mengurutkan data playlist lagu berdasarkan durasi dari yang terpendek hingga terpanjang.

Hasil pengujian menunjukkan bahwa algoritma Quick Sort mampu mengurutkan data dengan benar sesuai urutan yang diinginkan. Algoritma ini bekerja dengan membagi data menggunakan pivot dan mengurutkan setiap bagian secara rekursif, sehingga proses pengurutan menjadi lebih efisien dibandingkan beberapa algoritma sorting sederhana. Selain itu, penggunaan array sebagai struktur penyimpanan data memudahkan proses pengaksesan dan pertukaran elemen selama pengurutan berlangsung.

3.2 Saran

Berdasarkan hasil praktikum yang telah dilakukan, disarankan agar mahasiswa lebih banyak berlatih mengimplementasikan berbagai jenis algoritma sorting, seperti Shell Sort, Quick Sort, dan Merge Sort, sehingga dapat memahami perbedaan cara kerja, kelebihan, serta kekurangan masing-masing algoritma. Selain itu, mahasiswa perlu mencoba menguji program dengan jumlah data yang lebih banyak untuk melihat pengaruh ukuran data terhadap kinerja algoritma sorting.

Pada pengembangan selanjutnya, program dapat ditambahkan fitur input data secara dinamis, pencarian data lagu, serta pilihan pengurutan berdasarkan beberapa kriteria seperti judul, penyanyi, maupun durasi. Dengan demikian, program akan menjadi lebih interaktif dan mendekati

penerapan pada sistem pengelolaan data yang sebenarnya. Praktikum ini juga diharapkan dapat menjadi dasar untuk mempelajari struktur data dan algoritma yang lebih kompleks pada perkuliahan selanjutnya.

DAFTAR PUSTAKA

D4 Manajemen Informatika. (2025). *Algoritma Sort: Pengertian, Jenis, Cara Kerja, dan Contoh Lengkap*. Universitas Negeri Surabaya. Diakses pada 31 Mei 2026, dari <https://terapan-ti.vokasi.unesa.ac.id/post/algoritma-sort-pengertian-jenis-cara-kerja-dan-contoh-lengkap>